AicoreLink Landing Visual Redesign Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Port the visual design language of the static AicoreLink v2 site onto the six marketing sections that were explicitly scoped (Hero, Dolores, Planes, FAQ, CTA, Footer), without touching the `/herramientas/*` tool pages and without regressing any existing test invariant.

Architecture: The static site (`index.html` + `styles.css`) is the design source of truth. We translate its tokens into additive Tailwind design tokens, self-host its fonts through `next/font/local`, introduce one small shared "shell" module for the repeated section chrome (wrap / vertical rhythm / section tones / chapter eyebrow), and then restyle each of the six sections so that **every value still comes from `lib/marketing/copy.ts`** (the single source of truth). All existing anchors, `data-*` hooks, link hrefs, heading counts and plan counts are preserved so the current unit + e2e suites keep passing.

Tech Stack: Next.js 14.2 App Router · TypeScript 5.5 · Tailwind CSS 3.4 · `next/font/local` · vitest + React Testing Library · Playwright (Chrome channel).

Spec: `D:\aicorelink-landing-v2\aicorelink-landing\site\index.html` and `D:\aicorelink-landing-v2\aicorelink-landing\site\assets\css\styles.css` (the static design). Master context: `docs/superpowers/plans/2026-10-12-aicorelink-web-master-build-plan.md` (stage E1).

Global Constraints

- Work on branch `feat/landing-v2`. All commands run from `apps/dashboard`.
- **Scope is exactly six sections:** Hero, Dolores (pain), Planes (pricing), FAQ, CTA, Footer. **Do NOT touch** `app/(marketing)/herramientas/**` nor `components/marketing/{lead-cost-calculator,attention-test,demo-chat,voice-demo}.tsx`.
- `lib/marketing/copy.ts` is the **single source of truth** for every user-visible string. No marketing sentence may be hardcoded in JSX.
- All marketing strings are **Chilean Spanish, tuteo** (`tú`), **never voseo**.
- Prices are **CLP with IVA stated separately** (`+ IVA`) or `sin costo`. Never write a price inline; always via `formatCLP()`.
- Code, identifiers, comments, filenames, commit messages: **English**. Commit style: conventional commits, **no AI attribution**.
- **Preserve every existing test invariant** listed below (Task-by-task "Must keep" blocks).
- **Do not add runtime dependencies.** Fonts are self-hosted with `next/font/local` (no `next/font/google` — the Docker build has no guaranteed network access at build time).
- Keep `html` / `body` scroll behaviour, the `reveal` /IntersectionObserver pattern (no Framer Motion), and the `dynamic(..., { ssr:false })` lead-form loading.

- Anchors that MUST still exist: `#inicio #que-es #problema #solucion #como-funciona #rubros #planes #casos #preguntas #contacto`.

Token mapping (static CSS variable → new Tailwind token)

static var	value	new token	notes
`--navy`	`#0A2342`	`night`	dark section background
`--navy-deep`	`#061529`	`nightDeep`	deepest background (CTA)
`--navy-2`	`#123560`	`night2`	subtle dark surface
`--paper`	`#F3F6FA`	`paper`	light section background
`--ink`	`#12233D`	`ink`	text on light
`--ink-2`	`#44566F`	`inkMuted`	secondary text
`--line`	`#D5DDE8`	`hairline`	borders on light
`--cyan`	`#22D3EE`	`signal`	accent on dark
`--cyan-d`	`#08758A`	`signalDark`	accent on light
`--blue`	`#2F6BFF`	`action`	primary action
`--blue-d`	`#1D4FD1`	`actionDark`	primary action hover
`--amber`	`#F5A524`	`gold`	CTA button
`--amber-d`	`#C27803`	`goldDark`	CTA button hover
`--on-dark`	`#E6EEFB`	`onDark`	text on dark
`--on-dark-2`	`#A9BFDC`	`onDarkMuted`	secondary text on dark

> **Why semantic names and not `blue` / `cyan`:** `tailwind.config.ts` uses `theme.extend`, so built-in scales survive; naming a token `blue` or `cyan` would *override* Tailwind's `blue-500` / `cyan-400` scales, which the admin panel or other code may rely on. These names do not collide with any built-in Tailwind colour.

Review Focus

Five failure modes the spec implies but the existing tests do not exercise. Each has a test pinned in the task that owns the code.

1. **Narrow viewports with long Chilean copy.** A long headline/paragraph in a `clamp()`-sized font must not create horizontal overflow at 375px. → pinned in Task 2 (hero) and covered by the existing `responsive.spec.ts` no-overflow check.
2. **Reduced-motion users.** The marquee/ticker and any looping animation must be disabled under `prefers-reduced-motion: reduce`, and `.reveal` content must be visible. → pinned in Task 2.
3. **Font files missing at build/run time.** If the self-hosted `.woff2` are absent, the page must still render readable text through the `ui-sans-serif` / `Georgia` fallbacks (no crash, no layout collapse). → pinned in

Task 1.

4. **Billing toggle changing prices.** The Planes e2e asserts the exact strings `$19.990` / `$49.990` / `$99.990` are present; the **default** state must be Mensual so toggling to Anual never removes the monthly figures from the initial DOM. → pinned in Task 4.
5. **Accordion/keyboard accessibility on the new Dolores + FAQ markup.** Every expandable item must expose `aria-expanded` (FAQ) or a focusable native control (Dolores `<summary>`), and the section must still expose exactly the expected number of level-3 headings. → pinned in Tasks 3 and 5.

Task 1: Marketing design tokens, self-hosted fonts, section shell

Foundation task. Nothing renders differently yet on its own beyond the new font and token availability.

Files:

- Create: `apps/dashboard/app/fonts/figtree-latin-wght-normal.woff2`
- Create: `apps/dashboard/app/fonts/instrument-serif-latin-400-normal.woff2`
- Create: `apps/dashboard/app/fonts/instrument-serif-latin-400-italic.woff2`
- Create: `apps/dashboard/lib/marketing/shell.ts`
- Modify: `apps/dashboard/lib/design/tokens.ts`
- Modify: `apps/dashboard/tailwind.config.ts`
- Modify: `apps/dashboard/app/layout.tsx`
- Modify: `apps/dashboard/app/globals.css`
- Test: `apps/dashboard/__tests__/design-tokens.test.ts` (update)
- Test: `apps/dashboard/__tests__/marketing-shell.test.ts` (new)

Interfaces:

- Consumes: nothing.
- Produces:
  - `lib/design/tokens.ts` — `colors` gains keys `night, nightDeep, night2, paper, ink, inkMuted, hairline, signal, signalDark, action, actionDark, gold, goldDark, onDark, onDarkMuted` (all `string`); `fonts` becomes `{ sans: ["Figtree","ui-sans-serif","system-ui","sans-serif"]; display: ["Instrument Serif","Georgia","serif"] }`; `layout` gains `wrap: "mx-auto w-full max-w-[1160px] px-5"`.
  - `tailwind.config.ts` — `theme.extend.colors` gains one flat entry per new token above; `fontFamily` gains `sans: ["var(--font-figtree)", ...fonts.sans]` and `display: ["var(--font-instrument-serif)", ...fonts.display]` (so utilities `font-display`, `bg-night`, `text-signal`, `border-hairline`, … exist).
  - `lib/marketing/shell.ts` — exports:

```
export type SectionTone = "paper" | "white" | "night" | "nightDeep";
export const WRAP: string;                                  // layout.wrap
export const SECTION: string;                               // "py-[clamp(4rem,9vw,7.5rem)]"
export const TONE: Record<SectionTone, string>;             // bg + default text colour
export const CHAP: string;                                  // chapter eyebrow (Capítulo N)
export const HEAD: string;                                  // "max-w-[46rem]"
```

`TONE` exact values: `paper → "bg-paper text-ink"`, `white → "bg-white text-ink"`, `night → "bg-night text-onDark"`, `nightDeep → "bg-nightDeep text-onDark"`. `CHAP = "text-sm font-semibold tracking-wide text-signalDark"`.

- [ ] **Step 1: Copy the three font files**

Run (from the repo root):

```
New-Item -ItemType Directory -Force apps/dashboard/app/fonts | Out-Null
Copy-Item -LiteralPath 'D:\aicorelink-landing-v2\aicorelink-landing\site\assets\fonts\figtree-latin-
Copy-Item -LiteralPath 'D:\aicorelink-landing-v2\aicorelink-landing\site\assets\fonts\instrument-ser
Copy-Item -LiteralPath 'D:\aicorelink-landing-v2\aicorelink-landing\site\assets\fonts\instrument-ser
Get-ChildItem apps/dashboard/app/fonts
```

Expected: three `.woff2` files listed.

- [ ] **Step 2: Write the failing shell test**

Create `apps/dashboard/__tests__/marketing-shell.test.ts`:

```
import { describe, expect, it } from "vitest";
import { CHAP, HEAD, SECTION, TONE, WRAP } from "@/lib/marketing/shell";

describe("marketing shell", () => {
  it("exposes the section rhythm and container", () => {
    expect(WRAP).toContain("max-w-[1160px]");
    expect(SECTION).toContain("clamp(");
    expect(HEAD).toBe("max-w-[46rem]");
  });
  it("maps every tone to a background and text colour", () => {
    expect(TONE.paper).toBe("bg-paper text-ink");
    expect(TONE.white).toBe("bg-white text-ink");
    expect(TONE.night).toBe("bg-night text-onDark");
    expect(TONE.nightDeep).toBe("bg-nightDeep text-onDark");
  });
  it("styles the chapter eyebrow with the light-on-paper accent", () => {
    expect(CHAP).toContain("text-signalDark");
  });
});
```

- [ ] **Step 3: Run it to verify it fails**

Run: `npm run test -- __tests__/marketing-shell.test.ts` Expected: FAIL — `Failed to resolve import "@/lib/marketing/shell"`.

- [ ] **Step 4: Add the tokens**

In `apps/dashboard/lib/design/tokens.ts`, extend `colors` with the 15 new keys from the mapping table (exact hex, uppercase) and change `fonts` / `layout`:

```
fonts = { sans: ["Figtree", "ui-sans-serif", "system-ui", "sans-serif"], display: ["Instrument Serif
layout = { container: "max-w-6xl mx-auto px-4", sectionY: "py-20 md:py-28", wrap: "mx-auto w-full ma
```

Keep every existing key/value untouched. Widen the `ColorToken` / `ColorValue` types only if TypeScript requires it.

- [ ] **Step 5: Map the tokens in Tailwind and add the display family**

In `apps/dashboard/tailwind.config.ts`, inside `theme.extend.colors`, add one flat entry per new token (`night: colors.night`, …). Then:

```
fontFamily: {
  sans: ["var(--font-figtree)", ...fonts.sans],
  display: ["var(--font-instrument-serif)", ...fonts.display],
},
```

- [ ] **Step 6: Wire the self-hosted fonts in the root layout**

In `apps/dashboard/app/layout.tsx`, replace the `geist/font/sans` import with:

```
import localFont from "next/font/local";

const figtree = localFont({
  src: "./fonts/figtree-latin-wght-normal.woff2",
  variable: "--font-figtree",
  display: "swap",
  weight: "300 900",
});
const instrumentSerif = localFont({
  src: [
    { path: "./fonts/instrument-serif-latin-400-normal.woff2", weight: "400", style: "normal" },
    { path: "./fonts/instrument-serif-latin-400-italic.woff2", weight: "400", style: "italic" },
  ],
  variable: "--font-instrument-serif",
  display: "swap",
});
```

and set `<html lang="es" className={` ${figtree.variable} ${instrumentSerif.variable} `}>`. Keep the existing `metadata`, `metadataBase` and `<body className="bg-slate-50 font-sans text-slate-900 antialiased">`.

- [ ] **Step 7: Add the smooth-scroll + reduced-motion additions to globals.css**

Append to `apps/dashboard/app/globals.css` (keep everything already there):

```
@media (prefers-reduced-motion: reduce) {
  .ticker__track,
  .animate-float-slow {
    animation: none !important;
  }
}
```

(No `.ticker__track` exists yet — Task 2 defines it. Harmless now, required by Review Focus #2 once the ticker lands.)

- [ ] **Step 8: Update the design-tokens test expectations**

In `apps/dashboard/__tests__/design-tokens.test.ts`: update the `toEqual` on `colors` to include the 15 new keys, change `expect(fonts.sans[0]).toBe("Geist")` to `"Figtree"`, add `expect(fonts.display[0]).toBe("Instrument Serif")`, and add `expect(layout.wrap).toBe("mx-auto w-full max-w-[1160px] px-5")`. Keep the `resolveConfig(tailwindConfig)` assertions; add `expect(theme.fontFamily.display[0]).toBe("var(--font-instrument-serif)")` and `expect(theme.colors.night).toBe("#0A2342")`.

- [ ] **Step 9: Run the tests**

Run: `npm run test -- __tests__/marketing-shell.test.ts __tests__/design-tokens.test.ts` Expected: PASS.

- [ ] **Step 10: Typecheck, lint and build**

Run: `npm run typecheck` → clean. `npm run lint` → 0 errors (pre-existing warnings OK). `npm run build` → succeeds and logs the local font files as assets.

- [ ] **Step 11: Commit**

```
git add apps/dashboard/app/fonts apps/dashboard/lib/design/tokens.ts apps/dashboard/tailwind.config.
git commit -m "feat(marketing): add v2 design tokens, self-hosted fonts and section shell"
```

Task 2: Hero redesign (serif 2-line headline, rubro chips, 4-step thread, ticker)

Files:

- Modify: `apps/dashboard/lib/marketing/copy.ts` (the `hero` entry + new `ticker` entry)
- Modify: `apps/dashboard/components/marketing/hero-section.tsx`

- Create: `apps/dashboard/components/marketing/ticker.tsx`
- Test: `apps/dashboard/__tests__/hero-trustbar.test.tsx` (update)
- Test: `apps/dashboard/__tests__/landing-page.test.tsx` (only if the h1 assertion needs re-pointing — it should not)

Interfaces:

- Consumes: `WRAP`, `TONE.night` from `@/lib/marketing/shell` (Task 1).
- Produces: `copy.hero` gains `askLabel: string`, `micro: string`, `chatExampleLabel: string`, `chips: { id: string; label: string }[]` (5 items: `barberia/taller/abogados/inmobiliaria/clinica`), `thread: { label: string }[]` (4 items: `Escribe`, `Responde`, `Agenda`, `Pide reseña`), and a new `copy.ticker.items: string[]` (10 rubro labels). `HeroSection` keeps deriving the h1 from `copy.hero.headline`.

Must keep (existing assertions):

- Exactly one `role=heading level=1` and `toHaveTextContent(copy.hero.headline)`.
- `#inicio` present and visible; primary CTA → `#contacto` with `data-track="cta_click"`; ghost CTA → `#como-funciona`; the trust line text.

- [ ] **Step 1: Read the current hero test to learn its exact assertions**

Read `apps/dashboard/__tests__/hero-trustbar.test.tsx`. Note every string it asserts. If it asserts the *old* headline/eyebrow, those expectations change to the new copy values **but the behavioural assertions stay identical** (single h1, CTA hrefs, `data-track`, trust line).

- [ ] **Step 2: Extend the copy**

In `copy.ts`, set the hero fields to the static design's values (Chilean tuteo):

- `eyebrow`: `"Sitios web con IA para negocios que viven de atender clientes"`
- `headline`: `"Tu negocio atendiendo, aunque estés ocupado."`
- `askLabel`: `"Mira cómo atendería tu negocio:"`
- `micro`: `"Respondemos hoy. Sin compromiso y sin letra chica."`
- `chatExampleLabel`: `"Negocio de ejemplo"`
- `chips`: the five rubros above (labels `Barberías`, `Talleres`, `Abogados`, `Inmobiliarias`, `Clínicas`)
- `thread`: the four step labels above
- `subhead`, `primaryCta`, `secondaryCta`, `trustLine`: take the static design's text. Add `copy.ticker = { items: [...] }` with the 10 static rubro labels. **Derive the accent split at render time, not in copy:** the h1 renders `copy.hero.headline` split on its last sentence.

- [ ] **Step 3: Write the failing test for the accent split + reduced motion**

Add to `apps/dashboard/__tests__/hero-trustbar.test.tsx`:

```
it("renders the headline as one h1 with the accent phrase styled", () => {
  render(<HeroSection />);
  const h1 = screen.getByRole("heading", { level: 1 });
  expect(h1).toHaveTextContent(copy.hero.headline);
  expect(within(h1).getByText(/aunque estés ocupado\./).className).toContain("italic");
});
it("renders a chip per hero rubro", () => {
  render(<HeroSection />);
  expect(screen.getAllByRole("button").filter((b) => b.className.includes("chip")).length)
    .toBe(copy.hero.chips.length);
});
```

(Adjust the query to match the class/ `data-*` you actually use; the invariant is "one interactive chip per `copy.hero.chips` entry".)

- [ ] **Step 4: Run it to verify it fails**

Run: `npm run test -- __tests__/hero-trustbar.test.tsx` Expected: FAIL (no italic span / no chips).

- [ ] **Step 5: Rebuild the hero**

In `hero-section.tsx`:

- Section: `<section id="inicio" className={cn("relative isolate overflow-hidden", TONE.night)}>` with the existing decorative gradient/blur blobs, using `night` / `signal` / `action` colours instead of `navy` / `brand` / `teal`.
- Left column: eyebrow pill (dot in `signal`); `h1` in `font-display` (`text-[clamp(2.8rem,6.4vw,5.25rem)] leading-[1]`), split `copy.hero.headline` on the last sentence so the tail renders in `<span className="italic text-signal">`; `subhead`; `askLabel`; a `role="group"` chips row (one `<button>` per `copy.hero.chips`, with `aria-pressed`); CTAs via `buttonClasses("primary","lg")` → `#contacto data-track="cta_click"` and a ghost variant → `#como-funciona`; `micro` line.
- Right column: the chat mockup, restyled to the static `.chat` look (`bg-night2`, `chat__bar` with the "Responde en segundos" state, `copy.hero.chatExampleLabel`).
- Below the grid: a 4-step thread strip built from `copy.hero.thread` (numbered dots, `signal` active-state), then `<Ticker />`.
- **Never** introduce a second `h1`.

Create `ticker.tsx` as a Server Component: `role="marquee"`-free, `aria-hidden` on the duplicate set, a `.ticker__track` element animated with a CSS keyframe added to `globals.css` (`ticker-scroll`, `46s linear infinite`), paused on `hover` and **disabled** under `prefers-reduced-motion` (Task 1 Step 7 already covers that selector).

- [ ] **Step 6: Run the tests**

Run: `npm run test -- __tests__/hero-trustbar.test.tsx __tests__/landing-page.test.tsx` Expected: PASS (one h1, correct text, chips present).

- [ ] **Step 7: Commit**

```
git add apps/dashboard/lib/marketing/copy.ts apps/dashboard/components/marketing/hero-section.tsx ap
git commit -m "feat(marketing): redesign hero with serif headline, rubro chips and thread"
```

Task 3: Dolores redesign (inbox accordion)

Files:

- Modify: `apps/dashboard/lib/marketing/copy.ts` (`pains.items`)
- Modify: `apps/dashboard/components/marketing/pain-section.tsx`
- Delete: `apps/dashboard/components/marketing/pain-card.tsx` (folded into the section, or kept if still imported elsewhere — check first)
- Test: `apps/dashboard/__tests__/pain-section.test.tsx` (update)

Interfaces:

- Consumes: `WRAP`, `SECTION`, `TONE.paper`, `HEAD`, `CHAP` (Task 1).
- Produces: `copy.pains.items[i]` gains `message: string`, `time: string`, `whatHappened: string`, `withAicoreLink: string` while **keeping** `title` and `description`. `copy.pains` gains `chapter: string` (`"Capítulo 1"`), `inboxLabel: string` (`"Bandeja de ejemplo"`), `inboxTime: string` (`"Martes 22:14"`).

Must keep (existing assertions in `pain-section.test.tsx`):

- `section#problema`; an `h2` containing `copy.pains.title`; `copy.pains.closing` present.
- `getAllByRole("heading", { level: 3 })` length === `copy.pains.items.length` (**8**).
- Each `item.title` and each `item.description` present in the DOM.

- [ ] **Step 1: Write the failing test**

Add to `apps/dashboard/__tests__/pain-section.test.tsx`:

```
it("renders each pain as a native single-open details item", () => {
  const { container } = render(<PainSection />);
  const items = container.querySelectorAll('details[name="dolor"]');
  expect(items.length).toBe(copy.pains.items.length);
  items.forEach((el) => expect(el.querySelector("summary h3")?.textContent).toBeTruthy());
});
it("shows both the diagnosis and the AicoreLink outcome for every pain", () => {
  render(<PainSection />);
  copy.pains.items.forEach((item) => {
    expect(screen.getByText(item.whatHappened)).toBeInTheDocument();
    expect(screen.getByText(item.withAicoreLink)).toBeInTheDocument();
  });
});
```

- [ ] **Step 2: Run it to verify it fails**

Run: `npm run test -- __tests__/pain-section.test.tsx` Expected: FAIL (`details[name="dolor"]` count 0).

- [ ] **Step 3: Extend the copy**

In `copy.ts` fill `pains.items` with the 8 pains of the static design: keep the existing `title` / `description`, add `message` (the inbox preview line), `time` (e.g. `"22:14"`), `whatHappened`, `withAicoreLink`. Add `chapter`, `inboxLabel`, `inboxTime` to `copy.pains`.

- [ ] **Step 4: Rebuild the section**

In `pain-section.tsx`:

- `<section id="problema" className={cn(SECTION, TONE.paper)}>` → `<div className={WRAP}>` → `<div className={HEAD}>` with `<p className={CHAP}>{copy.pains.chapter}</p>` + `<h2>{copy.pains.title}</h2>` + `<p>{copy.pains.closing}</p>`.
- Two-column layout (`lg:grid-cols-2`) with a sticky intro on the left (per the static `.dolores`) and the inbox card on the right: a white rounded card with a header (`copy.pains.inboxLabel` / `copy.pains.inboxTime`).
- One `<details name="dolor" className="group ...">` per item. Its `<summary>` holds the avatar initial, an `<h3>{item.title}</h3>` (heading content **inside** `<summary>` is valid HTML), a `<span className="block">{item.message}</span>`, and `<span>{item.time}</span>` — **no** `<p>` **inside** `<summary>` (invalid). The panel holds `description`, then two cells "Qué pasó" (`item.whatHappened`) and "Con AicoreLink" (`item.withAicoreLink`, accent-bordered), revealed with `group-open:grid-rows-[1fr]` on a `grid grid-rows-[0fr]` wrapper + `overflow-hidden`.
- The `<h3>` must live in the always-visible `<summary>` so `getAllByRole("heading",{level:3})` still sees all 8 (closed `<details>` descendants are excluded from role queries by testing-library).
- Only one section `h2` ; exactly 8 `h3`.

- [ ] **Step 5: Run the tests**

Run: `npm run test -- __tests__/pain-section.test.tsx __tests__/landing-page.test.tsx` Expected: PASS.

- [ ] **Step 6: Commit**

```
git add apps/dashboard/lib/marketing/copy.ts apps/dashboard/components/marketing/pain-section.tsx ap
git commit -m "feat(marketing): redesign dolores as an inbox accordion"
```

Task 4: Planes redesign (serif cards, billing toggle, comparison table)

Files:

- Modify: `apps/dashboard/lib/marketing/copy.ts` (`pricing`)
- Modify: `apps/dashboard/components/marketing/pricing-section.tsx`

- Modify: `apps/dashboard/components/marketing/pricing-card.tsx`
- Create: `apps/dashboard/components/marketing/pricing-toggle.tsx` (`"use client"`)
- Test: `apps/dashboard/__tests__/pricing-section.test.tsx` (extend)
- Test: `apps/dashboard/e2e/responsive.spec.ts` (extend)

Interfaces:

- Consumes: `WRAP`, `SECTION`, `TONE.white`, `HEAD`, `CHAP` (Task 1).
- Produces: `copy.pricing` gains `chapter: string` (`"Capítulo 7"`), `billing: { monthlyLabel: string; annualLabel: string; annualTag: string; monthlySuffix: string; annualSuffix: string }` and `comparison: { caption: string; rows: { label: string; values: [string, string, string] }[] }`. `copy.pricing.plans` **stays 4 entries** (`semilla`, `starter`, `growth`, `elite`) with the existing fields.

Must keep (existing assertions in `pricing-section.test.tsx`):

- `section#planes`; `h2` = `copy.pricing.title`; every `plan.name`, `plan.forWho`, `plan.features`, `formatCLP(plan.priceCLP)`.
- `getAllByText("+ IVA")` length **3**; `getByText("sin costo")` present.
- Exactly one `[data-highlighted='true']`, and it is `growth`; badges `"Más elegido"` and `"Atención completa"`.
- Every CTA link (accessible name `plans[0].ctaLabel`) `href="#contacto"`, count === `plans.length`; `copy.pricing.note` present.
- **e2e:** `#planes article[data-plan]` count === 4; `$0`, `$19.990`, `$49.990`, `$99.990` all present; the highlighted card contains `"Más elegido"`.
- [ ] **Step 1: Write the failing test**

Add to `apps/dashboard/__tests__/pricing-section.test.tsx`:

```
it("defaults to monthly billing and never hides the monthly prices", () => {
  render(<PricingSection />);
  const monthly = screen.getByRole("radio", { name: copy.pricing.billing.monthlyLabel });
  expect(monthly).toBeChecked();
  copy.pricing.plans.forEach((plan) => {
    if (plan.priceCLP > 0) expect(screen.getByText(formatCLP(plan.priceCLP))).toBeInTheDocument();
  });
});
it("renders the comparison table rows", () => {
  render(<PricingSection />);
  copy.pricing.comparison.rows.forEach((row) => {
    expect(screen.getByText(row.label)).toBeInTheDocument();
  });
});
```

- [ ] **Step 2: Run it to verify it fails**

Run: `npm run test -- __tests__/pricing-section.test.tsx` Expected: FAIL (no billing radios / no comparison rows).

- [ ] **Step 3: Extend the copy**

Add `chapter`, `billing` and the 15-row `comparison` matrix (labels + the three plan columns) from the static `table.cmp`. Keep the four plans and their existing `priceNote` / `features` / `ctaLabel` values unchanged; only add `priceNoteAnnualCLP?` if the toggle needs an annual figure (optional — the static uses `plan__eq` text).

- [ ] **Step 4: Rebuild the section**

- `<section id="planes" className={cn(SECTION, TONE.white)}>` with `WRAP`, a `CHAP` eyebrow, the `h2` = `copy.pricing.title`, the subtitle, then `<PricingToggle />`, then the grid.
- Keep the grid rendering **all four plans** as `<PricingCard>` inside `<Reveal>`; the highlighted card must remain `article[data-plan="growth"][data-highlighted="true"]`.
- `pricing-card.tsx`: `<article data-plan data-highlighted className="...">` with a serif `h3` (`font-display text-[2.2rem]`), `plan.forWho`, a serif price (`formatCLP` + `/ mes` + `plan.priceNote`), the feature `ul` with `signalDark` checks, and the CTA `<a href="#contacto" data-track="cta_click" data-label={"planes-"+plan.id}>`. Keep the highlighted card visually elevated.
- `pricing-toggle.tsx`: `"use client"`, a `role="radiogroup"` with two `role="radio"` buttons (`aria-checked`), defaulting to **Mensual**; the annual option shows the `annualTag` badge. Toggling only changes suffix/`plan__eq` text — never unmounts the monthly prices (they stay in the DOM; you may dim them with an `aria-hidden` note but the numeric strings must remain).
- Below the grid add the comparison `<table>` from `copy.pricing.comparison` with a `<caption>`, then `copy.pricing.note`.

- [ ] **Step 5: Extend the e2e**

In `apps/dashboard/e2e/responsive.spec.ts` (inside the existing pricing test) add: the billing radiogroup exists, `Mensual` is checked by default, and after clicking `Anual` the four `article[data-plan]` are **still present** (count 4). Keep all existing pricing assertions.

- [ ] **Step 6: Run the tests**

Run: `npm run test -- __tests__/pricing-section.test.tsx` Expected: PASS.

- [ ] **Step 7: Commit**

```
git add apps/dashboard/lib/marketing/copy.ts apps/dashboard/components/marketing/pricing-section.tsx
git commit -m "feat(marketing): redesign pricing with billing toggle and comparison table"
```

Task 5: FAQ redesign (12 items, +/− marker)

Files:

- Modify: `apps/dashboard/lib/marketing/copy.ts` (`faq.items` 8 → 12)
- Modify: `apps/dashboard/components/marketing/faq-section.tsx`

- Modify: `apps/dashboard/components/ui/accordion.tsx` (marker styling only — keep the button/aria contract)
- Modify: `apps/dashboard/lib/marketing/structured-data.ts` (FAQ count will follow `copy.faq.items` automatically — verify)
- Test: `apps/dashboard/__tests__/faq-section.test.tsx` (update)
- Test: `apps/dashboard/__tests__/structured-data.test.ts` (update if it pins a count)

Interfaces:

- Consumes: `WRAP`, `SECTION`, `TONE.paper`, `HEAD` (Task 1).
- Produces: `copy.faq.items` length **12**; each item keeps `{ question, answer }`.

Must keep (existing assertions in `faq-section.test.tsx`):

- `section#preguntas`; `h2` = `copy.faq.title`; `copy.faq.subtitle`.
- One `role=button` per item with `aria-expanded`; the **first** item is `aria-expanded=true` by default and the rest `false`; clicking item `[3]` opens it **and closes item `[0]`**; the opened panel (`data-state="open"`) contains that item's answer.

- [ ] **Step 1: Update the test to the new count**

In `apps/dashboard/__tests__/faq-section.test.tsx`, replace any hardcoded `8` with `copy.faq.items.length` (test with 12 items; if item `[3]` / `[0]` indices are hardcoded they still work). Add:

```
it("renders twelve questions", () => {
  expect(copy.faq.items.length).toBe(12);
});
```

- [ ] **Step 2: Run it to verify it fails**

Run: `npm run test -- __tests__/faq-section.test.tsx` Expected: FAIL on the `12` expectation.

- [ ] **Step 3: Extend the copy to 12 items**

Add the four extra questions from the static design (keeping the existing eight, in the static order): the static list is — ¿Necesito saber de tecnología? / ¿Sirve para una tienda con catálogo? / ¿La IA puede decir algo incorrecto? / ¿Cuánto tardan? / ¿Funciona con mi WhatsApp actual? / ¿Qué pasa con los datos de mis clientes? / ¿Hay permanencia? / ¿Qué pasa si quiero irme? / ¿Entiende las notas de voz? / ¿Reemplaza mi caja, comisiones o boletas? / ¿Puedo probarlo antes de pagar? / ¿Y si no recibo consultas? Reuse existing answers where the question already exists; write tuteo answers for the new ones.

- [ ] **Step 4: Restyle**

- Section: `<section id="preguntas" className={cn(SECTION, TONE.paper)}>` with `WRAP` + a narrow column (`max-w-3xl`) and the `h2`/subtitle.
- `accordion.tsx`: keep the button/`aria-expanded`/`aria-controls`/`data-state` contract and the single-open behaviour exactly. Change only the marker: swap the chevron for a `+` → `–` indicator driven by `data-state` (or keep the chevron rotated — the test does not pin the icon). Ensure the question is still reachable as `role=button` with its text.

- [ ] **Step 5: Verify structured data follows**

Run: `npm run test -- __tests__/structured-data.test.ts __tests__/sitemap-robots.test.ts` Expected: PASS. If `structured-data` pins a question count, update it to `copy.faq.items.length`.

- [ ] **Step 6: Run the tests**

Run: `npm run test -- __tests__/faq-section.test.tsx` Expected: PASS.

- [ ] **Step 7: Commit**

```
git add apps/dashboard/lib/marketing/copy.ts apps/dashboard/components/marketing/faq-section.tsx app
git commit -m "feat(marketing): expand FAQ to twelve questions and restyle markers"
```

Task 6: CTA redesign (two columns + 3-step "after" list)

Files:

- Modify: `apps/dashboard/lib/marketing/copy.ts` (`cta`)
- Modify: `apps/dashboard/components/marketing/cta-section.tsx`
- Test: `apps/dashboard/__tests__/cta-section.test.tsx` (extend)

Interfaces:

- Consumes: `WRAP`, `SECTION`, `TONE.nightDeep`, `HEAD` (Task 1).
- Produces: `copy.cta` gains `steps: { label: string }[]` (3 items) and `formTitle: string` (`"Quiero mi diagnóstico gratis"`).

Must keep (existing assertions in `cta-section.test.tsx`):

- `section#contacto`; `h2` = `copy.cta.title`; `copy.cta.subtitle`; `copy.cta.guarantee`.
- The primary CTA (accessible name matching `copy.cta.primaryLabel`) has `href === bookingUrl`, `target="_blank"`, `rel="noopener noreferrer"`.
- The secondary CTA has `href === whatsappUrl(copy.stickyWhatsApp.message)`, contains `WHATSAPP_PHONE`, `target="_blank"`, `rel="noopener noreferrer"`.
- `await screen.findByRole("button", { name: /Solicitar diagnóstico gratis/ })` resolves (lazy lead form still mounted).
- **e2e:** `a[data-label="agendar-diagnostico"]` visible and not overlapped by the sticky WhatsApp.

- [ ] **Step 1: Write the failing test**

Add to `apps/dashboard/__tests__/cta-section.test.tsx`:

```
it("lists the three next steps", () => {
  render(<CtaSection />);
  copy.cta.steps.forEach((step) => expect(screen.getByText(step.label)).toBeInTheDocument());
});
```

- [ ] **Step 2: Run it to verify it fails**

Run: `npm run test -- __tests__/cta-section.test.tsx` Expected: FAIL.

- [ ] **Step 3: Extend the copy** with `steps` (3 short tuteo labels, e.g. "Nos cuentas de tu negocio" / "Te mostramos el diagnóstico" / "Sales atendiendo") and `formTitle`.
- [ ] **Step 4: Rebuild the section**
- `<section id="contacto" className={cn("relative isolate overflow-hidden", TONE.nightDeep)}>` (scrollable, `text-onDark`).
- Two columns (`lg:grid-cols-2`): left = `h2` (serif), `subtitle`, the primary + secondary CTAs **with their exact existing hrefs/attrs**, the guarantee line; right = the lazily-loaded `<LeadForm />` inside a `<Reveal>` with `formTitle` as its heading.
- The `data-label="agendar-diagnostico"` attribute must stay on the primary link (e2e depends on it).
- Do not change `lead-form.tsx` internals — its submit button text `"Solicitar diagnóstico gratis"` is asserted.
- [ ] **Step 5: Run the tests**

Run: `npm run test -- __tests__/cta-section.test.tsx` Expected: PASS.

- [ ] **Step 6: Commit**

```
git add apps/dashboard/lib/marketing/copy.ts apps/dashboard/components/marketing/cta-section.tsx app
git commit -m "feat(marketing): redesign final CTA with next-steps list"
```

Task 7: Footer redesign (4-column deep grid)

Files:

- Modify: `apps/dashboard/lib/marketing/copy.ts` (`footer`)
- Modify: `apps/dashboard/components/marketing/site-footer.tsx`
- Test: `apps/dashboard/__tests__/site-footer-integration.test.tsx` (extend)

Interfaces:

- Consumes: `WRAP`, `TONE.nightDeep` (Task 1).
- Produces: `copy.footer` gains `navTitle: string` (`"AicoreLink"`) and `navLinks: { label: string; href: string }[]` (the four section anchors `#dolores / #calculadora / #planes / #faq` → map to the real anchors `#problema / #herramientas/calculadora / #planes / #preguntas`). Existing fields (`tagline`, `ecosystem{title,links}`, `contact{title,...}`, `legalLinks`, `legal`) stay.

Must keep (existing assertions in `site-footer-integration.test.tsx`):

- `copy.brand.name`; `copy.footer.tagline`; `copy.footer.ecosystem.title` + every ecosystem link (href, `target="_blank"`, `rel="noopener noreferrer"`); `copy.footer.contact.title`; contact link[0] (WhatsApp, `data-label="footer"`, `target` / `rel`); contact link[1] `href="#contacto"`;

`getByText("Legal")` (the exact string must remain as a group title); every `copy.footer.legalLinks` href; `copy.footer.legal` copyright text.

- [ ] **Step 1: Write the failing test**

Add to `apps/dashboard/__tests__/site-footer-integration.test.tsx`:

```
it("renders the four-column layout with an AicoreLink nav column", () => {
  render(<SiteFooter />);
  expect(screen.getByText(copy.footer.navTitle)).toBeInTheDocument();
  copy.footer.navLinks.forEach((link) => {
    expect(screen.getByRole("link", { name: link.label })).toHaveAttribute("href", link.href);
  });
});
```

- [ ] **Step 2: Run it to verify it fails**

Run: `npm run test -- __tests__/site-footer-integration.test.tsx` Expected: FAIL.

- [ ] **Step 3: Extend the copy** with `navTitle` + `navLinks` (four entries pointing at anchors that exist on the page).
- [ ] **Step 4: Rebuild the footer**
- `<footer className={cn("border-t border-white/10", TONE.nightDeep)}>` with `WRAP` and a `md:grid-cols-4` grid: (1) brand mark + `copy.brand.name` + `copy.footer.tagline` + `copy.brand.tagline`; (2) `copy.footer.ecosystem.title` + links; (3) `copy.footer.contact.title` + contact links (keep `data-label="footer"` on the WhatsApp one); (4) a column whose title is **exactly** `Legal` + `copy.footer.legalLinks`. Put `copy.footer.navTitle` / `navLinks` under the brand column (or as a fifth `md:col-span` if the grid allows) — the assertion only requires the heading text to be present.
- Bottom bar: the copyright line (`copy.footer.legal` with the current year) — keep it present.
- [ ] **Step 5: Run the tests**

Run: `npm run test -- __tests__/site-footer-integration.test.tsx` Expected: PASS.

- [ ] **Step 6: Commit**

```
git add apps/dashboard/lib/marketing/copy.ts apps/dashboard/components/marketing/site-footer.tsx app
git commit -m "feat(marketing): redesign footer as a four-column deep grid"
```

Task 8: Page assembly, SEO consistency and copy-fit audit

Files:

- Modify: `apps/dashboard/app/(marketing)/page.tsx`
- Modify: `apps/dashboard/lib/marketing/site.ts` (only if the description must mention the new pricing framing)
- Test: `apps/dashboard/__tests__/landing-page.test.tsx`

- Test: `apps/dashboard/__tests__/marketing-copy.test.ts`, `apps/dashboard/__tests__/site-metadata.test.ts`, `apps/dashboard/__tests__/structured-data.test.ts`

Interfaces:

- Consumes: all six redesigned sections; `copy` (SSOT).
- Produces: unchanged public exports; the page keeps the same section order and the JSON-LD script.

Must keep:

- Anchors `que-es, problema, solucion, como-funciona, rubros, planes` exist and appear in the order `que-es → como-funciona → rubros → planes → contacto`.
- Exactly one `h1` (the hero), text === `copy.hero.headline`.
- The JSON-LD script is still emitted via `serializeStructuredData()`.
- `e2e/responsive.spec.ts`: all ten `SECTIONS` ids visible, no horizontal overflow, zero console errors.
- [ ] **Step 1: Fit the copy**

Run the full copy test to see every string the suite pins, then reconcile:

```
npm run test -- __tests__/marketing-copy.test.ts
```

Update any assertion that depended on the old hero/FAQ/pricing wording to the new values — **never delete an assertion**, re-point it.

- [ ] **Step 2: Reconcile SEO/JSON-LD counts**

Run: `npm run test -- __tests__/structured-data.test.ts __tests__/site-metadata.test.ts` Expected: PASS. If the FAQ schema count is asserted, it must now be 12 and derived from `copy.faq.items`.

- [ ] **Step 3: Verify the page assembly**

Open `app/(marketing)/page.tsx`; confirm the ten section components are still mounted in the same order and the JSON-LD script is present. Adjust the wrapper classes only where the new `SECTION` / `TONE` shells require it (the sections now carry their own background, so remove any conflicting `bg-*` from the page wrapper).

- [ ] **Step 4: Run the full unit suite**

Run: `npm run test` Expected: **all green** (was 487 tests / 56 files). Any failure is a regression to fix, not to skip.

- [ ] **Step 5: Commit**

```
git add apps/dashboard/app apps/dashboard/lib/marketing/copy.ts apps/dashboard/lib/marketing/site.ts
git commit -m "feat(marketing): assemble redesigned landing and align SEO copy"
```

Task 9: E2E, Lighthouse and deploy

Files:

- Modify: `apps/dashboard/e2e/responsive.spec.ts` (only additive assertions)
- Create: `apps/dashboard/e2e/redesign.spec.ts` (new section-specific assertions)
- Modify: `.superpowers/sdd/` ledger + a task report (gitignored)

Interfaces:

- Consumes: the whole redesigned page.
- Produces: a green Playwright run across 375/768/1440 and a Lighthouse report.

- [ ] **Step 1: Extend the responsive spec**

Keep every existing assertion. Add to `responsive.spec.ts`:

```
test("redesigned sections render their key chrome", async ({ page }) => {
  await page.goto("/");
  await expect(page.locator('#inicio h1')).toHaveCount(1);
  await expect(page.locator('#problema details[name="dolor"]')).toHaveCount(8);
  await expect(page.locator('#planes article[data-plan]')).toHaveCount(4);
  await expect(page.locator('#preguntas [role="button"]')).toHaveCount(12);
  await expect(page.locator('body')).not.toHaveCSS("overflow-x", "scroll");
});
```

Create `e2e/redesign.spec.ts` for the billing toggle (click `Anual`, assert 4 plans still present) and the Dolores single-open behaviour (open a second item, assert the first closed) — iterations that are awkward to keep in `responsive.spec.ts`.

- [ ] **Step 2: Build and run e2e**

Run: `npm run build` then, from `apps/dashboard`, `npm run start -- --port 3111` (background), wait for `http://localhost:3111/`, then `npm run test:e2e`. Expected: **all e2e green** across the three viewports (existing + new specs).

- [ ] **Step 3: Lighthouse**

Run the existing Lighthouse config against the production build on 3111 and compare to the documented baseline (**A11y 100 / BP 100 / SEO 100**, Perf ≈ 91-96). Expected: no regression below the baseline. Investigate any a11y drop (the redesigned headings/contrast are the likely cause).

- [ ] **Step 4: Kill the test server and record the outcome**

Kill the listener on 3111 (`Get-NetTCPConnection -LocalPort 3111 | Stop-Process -Force`). Append a task report to `.superpowers/sdd/2026-10-13-aicorelink-factory-plan/` (or a new landing-redesign ledger) with: files touched, gate results, Lighthouse numbers and any accepted deviation.

- [ ] **Step 5: Commit**

```
git add apps/dashboard/e2e
git commit -m "test(marketing): cover the redesigned landing in e2e"
```

- [ ] **Step 6: Merge and deploy**

Follow the `finishing-a-development-branch` flow: full gates green → merge `feat/landing-v2` → `main` (production deploys `main`) → push → verify `https://aicorelink.aicorebots.com` returns 200 and shows the redesigned hero.

Self-Review

1. Spec coverage. Every static section named in the agreed scope maps to a task: Hero (2), Dolores (3), Planes (4), FAQ (5), CTA (6), Footer (7); the design tokens/fonts/rhythm (1); the assembly/SEO consequences (`faq` → JSON-LD) (8); verification/deploy (9). Out-of-scope sections (calculadora/test/verticales/como-funciona/garantía/nosotros) are explicitly excluded and remain as they are.

2. Step scan. Each step names one action: copy a file, write an assertion, run a command, edit a named file. No step says "handle edge cases" or "add validation".

3. Type consistency. `SectionTone` / `TONE` / `WRAP` / `SECTION` / `HEAD` / `CHAP` are defined once in Task 1 and consumed by Tasks 3-7 by those exact names. `copy.pricing.billing.*`, `copy.cta.steps`, `copy.footer.navLinks`, `copy.hero.chips`, `copy.hero.thread`, `copy.pains.*` extra fields are defined where produced and referenced verbatim where consumed. `copy.pricing.plans` stays four entries across Tasks 4 and 9.

4. Review Focus. Narrow-viewport overflow → Task 2 + the existing no-overflow e2e. Reduced motion → Task 1 Step 7 + Task 2 ticker. Missing font files → Task 1 Steps 4-6 (fallbacks declared in `fonts`). Billing toggle hiding prices → Task 4 Steps 1 and 5. Accordion a11y/heading counts → Task 3 Step 4 and Task 5 Step 4.

5. Proportion. Nine tasks for six redesigned sections plus foundation and verification; the plan carries signatures, copy shapes and assertions rather than full component bodies.